

Linux Outage Triage: 20 Commands I Run Before I Touch Anything

[Ashish Singh](#)



When an outage hits at 3 AM, the fastest way to lose time is to start “trying things.”

Random restarts. Random config edits. Random scaling.

They often make the incident **worse**.

After 5 years of on-call rotations, I’ve learned that real

incident response isn't about being clever — it's about being **systematic**.

In every outage, I'm racing to answer **three questions**:

***What's broken?** (symptoms)*

***Where is it broken?** (blast radius)*

***What changed?** (direction to root cause)*

This is my on-call command list — **copy/paste ready**, grouped by purpose.

If you only read one section, read **"First 3 Minutes"**.

The First 3 Minutes (Copy/Paste This Block)

When the alert fires, I run this **exact sequence**:

```
uptime
top -o %CPU
free -h
df -h && df -i
ss -tulpn
journalctl -xe --no-pager | tail -n 80
```

What this tells you in 60 seconds

- ✓ **uptime** → Load average + system stability
- ✓ **top** → Top CPU offenders (is CPU actually the problem?)
- ✓ **free -h** → Memory pressure / swap thrashing
- ✓ **df -h / df -i** → Disk full **OR** inode exhaustion (both kill systems)
- ✓ **ss -tulpn** → What's listening + which ports exist (or don't)
- ✓ **journalctl -xe** → Last critical errors the system is screaming about

These 6 commands give you the incident context in under a minute.

Now let's go deeper.

Section 1: Is the Host Dying? (CPU / Load)

```
uptime
top -o %CPU
ps aux --sort=-%cpu | head -n 15
mpstat -P ALL 1
pidstat 1
```

Use this when: Everything feels slow, requests timeout, CPU alerts are firing.

What to look for:

- Load average **way higher** than CPU core count
- One process eating CPU continuously
- CPU looks fine but system is slow → often **IO wait** (jump to disk section)

Real example: Load average of 47 on an 8-core box. Turned out to be a runaway Python script consuming 780% CPU (multi-threaded). `pidstat` showed the exact PID.

Section 2: Memory Pressure / OOM (The Invisible Killer)

```
free -h
ps aux --sort=-%mem | head -n 15
dmesg -T | tail -n 120
journalctl -k --no-pager | tail -n 120
```

Use this when: Services restart randomly, pods/daemons crash, the box feels unstable.

What to look for:

- OOM killer logs: "Killed process 12345 (java) ..."
- Swap usage climbing fast (80%+ is danger zone)
- One process ballooning in memory

Critical rule: Capture evidence **first** (logs + memory snapshot) before restarting anything.

Why? Once you restart, the evidence is gone. You'll never know what actually happened.

Section 3: Disk Full OR Inode Full (Both Cause Outages)

```
df -h
```

```
df -i
```

```
du -sh * 2>/dev/null | sort -h | tail -n 15
```

Use this when: Writes fail, apps crash, databases misbehave, deployments hang.

What to look for:

- Disk usage at **95–100%**
- Inodes at **100%** (common with millions of tiny log files)

Pro tip: `df -i` is the command **everyone forgets**. I've seen 3-hour outages caused by inode exhaustion while disk was only 60% full.

Section 4: Deleted Files Still Eating Disk (Sneaky Outage)

```
lsdf +L1
```

Use this when: `df -h` says disk is full but you "deleted 50GB of logs" and space didn't come back.

Why it happens: A process is still holding the deleted file open.

Fix direction: Restart *that specific process* (after collecting evidence).

Real story: Once spent 2 hours debugging disk space. Logs showed 89% full. Deleted 40GB. Still 89%. Ran `ls -l` → found a Java process holding a deleted 45GB file. Restart fixed it instantly.

Section 5: Disk IO Bottleneck (Slow System, Fine CPU)

```
iostat -xz 1  
vmstat 1
```

Use this when: Latency spikes, hosts feel frozen, but CPU isn't pegged.

What to look for:

- High disk **await** times (>50ms is bad, >100ms is crisis)
- `vmstat` showing **IO wait** / blocked processes

This is the #1 misdiagnosed issue. People see “system slow” and assume CPU. Often it’s disk IO.

Section 6: What’s Listening? (Ports + Services)

```
ss -tulpn  
ss -s
```

Use this when: “Connection refused,” health checks fail, traffic can’t reach a service.

What to look for:

- The service port **is not listening** (common after failed deploy)
- Too many sockets / connection pressure

Section 7: Active Connections (Is Traffic Melting You?)

```
ss -tan state established | wc -l  
ss -tan | head -n 30
```

Use this when: Sudden traffic surge, connection storms, unexpected client behavior.

What to look for:

- Huge number of established connections (>10K on a single host)
- Many connections in weird states (can indicate upstream issues)

Section 8: Basic Network Truth (Interfaces + Routes)

```
ip a  
ip r
```

Use this when: Packets don't go where you expect, cross-host calls fail.

What to look for:

- Missing interface / wrong IP assignment
- Wrong default route
- Route changes after a restart or config update

Section 9: Quick Reachability Tests (Don't Overthink)

```
ping -c 3 <target>  
curl -I http://<service>  
curl -vk https://<service>
```


Use this when: You need to confirm "is it network or application?"

Pro tip: `curl -vk` is **gold** for TLS debugging (cert chain, handshake failures, redirects, SNI issues).

Section 10: Service Health (systemd)

```
systemctl status <service>
systemctl is-active <service>
systemctl restart <service>
```

Use this when: You suspect the service itself is down or crash looping.

Golden rule: Read logs **first** before restarting. Otherwise you're just guessing.

Section 11: Service Logs (The "Why")

```
journalctl -u <service> --since "30 min ago" --no-pager
journalctl -u <service> -f
```

Use this when: You need root cause direction (config errors, dependency failures, auth errors).

Tip: Use `--since "30 min ago"` to reduce noise. Most outages have signals in the last 30 minutes.

Section 12: Recent System Errors (Kernel + System)

```
journalctl -p err..alert --since "1 hour ago" --no-pager  
dmesg -T | tail -n 120
```

Use this when: You suspect kernel/disk/network driver issues or weird host behavior.

What this catches: OOM kills, kernel panics, driver failures, hardware errors.

Section 13: What Changed? (Config Drift / Timestamps)

```
ls -ltr /etc/<app>/  
stat /etc/<app>/<config>
```

Use this when: The outage started “after something,” but no one remembers what.

Why it matters: 80% of outages are caused by **changes**. Find the change, find the cause.

Section 14: Search Logs for Real Signals

```
grep -R "ERROR\|FATAL\|panic\|exception" -n /var/log
```

Use this when: You have too much log noise and need the error line **now**.

Alternative: `journalctl -p err` for systemd-managed services.

Section 15: Process Ownership (What Spawned This?)

```
ps aux | grep <process_name>  
pstree -ap | head -n 40
```

Use this when: You need to see parent/child relationships (especially for runaway processes).

Example use case: A zombie process that keeps respawning. `pstree` shows the parent that needs to be killed.

Section 16: Who Changed Files Recently?

```
find /etc/<app>/ -type f -mmin -120 -ls 2>/dev/null
```

Use this when: The config directory is large and you need "what changed in the last 2 hours?"

Pro move: Combine with `stat` to see exact modification time + who changed it.

Section 17: DNS Checks (Because Yes... It's Often DNS)

```
resolvectl status 2>/dev/null || cat /etc/resolv.conf  
dig +short <domain>  
nslookup <domain>
```

Use this when: Services can't resolve dependencies, intermittent failures across nodes.

Fun fact: In my experience, 30% of "mysterious network issues" are DNS. Check it early.

Section 18: Trace a Request Path (Quickly)

```
traceroute <target> 2>/dev/null || tracepath <target>
```

Use this when: Traffic disappears between hosts/VPCs/subnets.

What this shows: Where packets are dropping (firewall rules, routing black holes, VPC peering issues).

Section 19: File Descriptor Pressure (Too Many Open Files)

```
ulimit -n  
lsof | wc -l
```

Use this when: Strange IO failures, "too many open files" errors, crashes under load.

Common cause: Application not closing connections/files properly. Check application code.

Section 20: Verify Time (Time Drift Breaks Auth + TLS)

```
timedatectl status  
date
```

Use this when: Tokens fail, TLS errors appear, distributed systems behave oddly.

Why it matters: Clock drift >5 seconds can break:

- JWT token validation

- TLS certificate validation
- Distributed consensus (Raft, Paxos)
- Time-series databases

Common Mistakes That Make Incidents Worse

I've made every one of these mistakes. Learn from my pain:

1. Restarting before collecting logs/evidence → You lose the smoking gun. Always capture first.

2. Ignoring `df -i` (inodes) → Disk can be 60% full but system fails because inodes are 100%.

3. Assuming "CPU high" means compute → Often it's IO wait masquerading as CPU usage.

4. Debugging one node without checking blast radius → Is it one host or all hosts? Find out in the first minute.

5. Fixing symptoms without finding what changed → The outage will come back. Find the root cause.

Quick Reference: Outage Command Map (1-Minute Guide)

Scenario: Users seeing timeouts

```
uptime + top + ss -tulpn + journalctl -u <service>
```

Scenario: Everything is slow but CPU looks fine

```
iostat -xz 1 + vmstat 1
```

Scenario: Writes fail / deploy fails

```
df -h + df -i + lsof +L1
```

Scenario: Service is “up” but still broken

```
journalctl -u <service> + grep errors + stat /etc/<service>
```

The Real Lesson

Outage response isn't about being the smartest person in the room.

It's about being **systematic**.

These 20 commands are my system.

Copy them. Paste them. Build your own runbook.

And when the alert fires at 3 AM, you'll know exactly where

to start.

What's Next?



Like & Share if you found this useful!



Follow for more practical DevOps playbooks.



Enjoyed this article? Give it 50 claps!

[Lets Connect On Linkedin](#)

And if you've got your own war stories or commands I missed — drop them in the comments. I'm always learning.